# Ders 3 Çalışma Özeti

## Ders 3 Çalışma Özeti

### Genel Konular

- Yazılım proje planı: yaşayan plan kavramı ve planın içeriği.
- Yazılım geliştirme planının adımları: sorunun tanımlanması, çözüm yolunun kararlaştırılması, geliştirme sürecinin planlanması, proje organizasyonu, proje zamanlaması.
- İş kırılım yapısı (Work Breakdown Structure, WBS) örnekleri.
- Alternatif fizibilite: teknik, ekonomik ve kullanılabilirlik fizibilitesi.
- Yazılım geliştirme süreç modelinin seçimi.
- Proje planının ayrıntılı içerik listesi: kapsam, süreç modeli, varsayım ve kısıtlar, risk analizi, alt aktiviteler, zaman çizelgesi, iş gücü, görev atama, bütçe, kalite planı.
- Yazılım risk yönetimine giriş ve risk kategorileri.
- Makale önerileri üzerine geri bildirimler ve teklif teslim duyurusu.

### Hocanın Özellikle Vurguladığı Kısımlar

- Yazılım proje planı yaşayan bir plan olmalıdır. Koşullar değiştikçe güncellenmelidir. Bu yüzden dinamik planlamaya izin veren proje planlama ortamları kullanılmalıdır.
- Plan çok uzun ve karmaşık olmamalıdır. Net izlenebilir bir yapıda olmalıdır.
- Sorunun tanımlanmasında müşteriyi ilgilendirmeyen yazılım tekniği sorunlarına girilmez. Sorun müşterinin anlayacağı terimlerle tanımlanır.
- Kabul kriterleri baştan net konursa kabul muayenesinde müşteriyle anlaşmazlık çıkmaz.
- Çözüm yolu tek başına proje yöneticisinin kararı olmamalıdır. Yazılım mimarı ve tasarımcılarla tartışılarak verilmelidir.
- Bir çözüm yolu ancak sağlanan kaynaklar ve teknoloji düzeyiyle verilen sürede gerçekleştirilebiliyorsa yapılabilirdir.
- En iyi teknik çözüm kurum çalışanlarına kullanılabilir gelmiyorsa en iyi çözüm değildir.
- Seçilmeyen çözüm yollarının red nedenleri belgelenmelidir. Üretimin öncelik sırası da listelenmelidir.
- WBS, seçilen yazılım geliştirme süreç modeline göre oluşturulur. Bu yüzden model önce seçilmelidir.
- Makale konusu, dersin genel konuları (risk yönetimi, yazılım ölçümü, maliyet tahmini) olmamalıdır. Bu konulardaki yeni açılımlar ve yaklaşımlar seçilmelidir.

### Kısa Tekrar Notları

- Yazılım proje planının temel soruları: Ne yapılacak? Ne kadar kaynakla? Ne kadar zamanda?
- Plan içeriği (minimum):
  1. Konu: amaç, hedef, ana işlevler, diğer özellikler, geliştirme süreci.
  2. Kaynaklar: insan, donanım, yazılım, zaman.
  3. Risk yönetimi.
  4. Gider (bütçe).
  5. Ek çizelgeler: görev ağı planı, zamanlama çizelgesi, iş-kaynak çizelgesi.
- Gereksinim: bir sorunun çözümü için sistemden beklenen yetenek.
- WBS: proje işlerinin alt başlıklar hâlinde hiyerarşik gösterimi. Numaralı liste veya diyagram olabilir.

- Alternatif fizibilite: Her çözüm yolu teknik, ekonomik ve kullanılabilirlik açısından karşılaştırılır.
- Ekonomik fizibilite yalnızca maliyet değil, getiriyi de içerir.
- İş gücü belirleme: “15 gün çalışacak 3 sistem analisti gerekir.” Görev atama: “Sistem analisti olarak şu kişiler atanır.”
- Risk: şu anda olmayan ama gelecekte projeyi kötüye götürebilecek problem. Belirsizlik ve kayıp özelliği vardır.
- Risk kategorileri: ürün büyüklüğü, işletmecilik, müşteri, süreç, teknoloji (teknik), geliştirme ortamı, personel.
- Aynı kategoriler varsayım ve kısıtları belirlemek için de kullanılabilir.

## Detaylı Açıklamalar

### Yazılım proje planı

- Proje yöneticisi, üstlendiği görev için kaynakları, görevin nasıl ve hangi sürede tamamlanacağını belirler.
- Plan, deneyime dayalı tahminlerle yapılır. Görev ilerledikçe yeniden gözden geçirilir ve düzeltilir.
- Farklı plan formatları vardır. IEEE standartları her aşama için belge formatı verir: gereksinim belgesi (SRS), proje planı standardı, konfigürasyon dokümanı standardı gibi.
- Zaman, kaynaklar altında da ayrı bir madde olarak da yazılabilir. Ek çizelgelerin çoğu zamana bağlıdır.

### Yazılım geliştirme planının adımları

1. Sorunun tanımlanması.
2. Çözüm yolunun kararlaştırılması.
3. Geliştirme sürecinin planlanması.
4. Proje organizasyonu.
5. Proje zamanlaması.

- Bunların içinde kabul kriterleri, kısıtlar, varsayımlar ve riskler de belirlenir.

### Sorunun tanımlanması

- Müşteri talebi varsa sorun müşteri istekleri yönünden tanımlanır. Genel bir ürünse ürüne neden ihtiyaç duyulduğu, piyasadaki yeri ve getirisi düşünülür.
- Müşterinin amacı ve hedefi nedir? Ör. bordro hesabı, envanter, trafik kontrolü.
- Eski bir yazılım varsa sorunları neler, yeniden beklentiler neler?
- Müşteriden alınan bilgilere göre plan gözden geçirilir. Olay anlaşıldıkça hedeflerde değişiklik olup olmadığı kontrol edilir.
- Koşullar ve kısıtlar belirlenir. Belirli bir müşteriye özgü kısıtlar da yazılır.
- Gereksinimler genel olarak tanımlanır: istenen işlevler, sistem yetenekleri, donanım, yazılım ve kullanıcı arabirimleri, uyulması istenen standartlar. Ayrıntılı gereksinim analizi plandan sonra yapılır.
- Müşterinin kabul kriterleri belirlenir.
- Kabuller belirlenir: hem işleyiş (fonksiyonel) hem olanaklar açısından.
- Amaçlara ulaşma mekanizması belirlenir: geliştirme sürecinin dönüm noktaları ve kilometre taşları (gereksinim analizi, tasarım, test, bakım), alt işlemler ve zamanlaması.
- Yazılım mühendisi sorunun bilgisayar desteğiyle çözüm olanağını belirler: donanım, yazılım ve kullanıcı öğelerinin işlevleri, kısıtları ve uyumluluğu.
- Sistem düzeyinde amaç, hedef, gereksinim ve kabul kriterleri konur.

### İş kırılım yapısı örneği

```
1.0 Analiz
  1.1 Bilgi toplama
  1.2 Modelleme
  1.3 SRS belgesinin hazırlanması
```

```
2.0 Tasarım
  2.1 Ön tasarım
  2.2 Ayrıntılı tasarım
```

- Aynı yapı diyagram olarak da gösterilebilir. Önemli olan hiyerarşinin gösterilmesidir.
- Türkçede iş kırılım yapısı, hiyerarşik iş listesi gibi adlarla da anılır.

**Çözüm yolunun kararlaştırılması**

- Sorun anlaşıldıktan sonra olası çözüm yolları genel olarak açıklanır. Bu aşamada ayrıntılı plan beklenmez.
- Dikkate alınanlar:
  - Kullanıcıların karşılaşabileceği dış faktörler.
  - Geliştiricilerin bilgisi: Hangi yoldan gidilirse çalışanlar güçlü oldukları alanda hızlı üretim yapar?
- Çözüm yolu, tasarımcıya seçim olanağı tanıyan alternatifler hâlinde ifade edilir.
- Her çözüm yolunun sınırları ve kısıtları belirlenir. Ör. en iyi çözüm için eğitim gerekiyorsa buna zaman var mı?
- Seçilen çözüm yolu tasarım ve gerçekleştirme için çerçeve oluşturur.

**Alternatif fizibilite**

- Her çözüm alternatifi üç açıdan değerlendirilir:
  - **Teknik fizibilite:** Teknik altyapı, çözüm yolu, yazılım geliştirme süreci ve modeli.
  - **Ekonomik fizibilite:** Her çözümün maliyeti ve getirisi. İkisi karşılaştırılır.
  - **Kullanılabilirlik fizibilitesi:** Ürün son kullanıcıya uygun mu? Ekran yapıları, ekran geçişleri, seçilen donanım uygun mu?
- Değerlendirme yöntemleri: olumlu ve olumsuz yönler, güçlü ve zayıf yönler, karşılaştırma tablosu, en kötü durum analizi.
- Sistemin nasıl davranacağı kestirilemiyorsa simülasyon veya model oluşturulabilir.
- Sonuçta bir alternatif fizibilite matrisi oluşur. Bir seçenek seçilir, diğerlerinin red nedenleri açıklanır.
- Önerilen çözüm yolunda üretimin öncelik sırası liste olarak verilir. Uyumsuzluk, darboğaz veya bütçe aşımı nedeniyle bazı ürünlerden vazgeçmek gerekirse bu liste kullanılır. Öncelik sırası yoksa kararlar zayıf kalır.

**Geliştirme sürecinin planlanması**

- Kabaca gereksinim analizi yapıldıktan sonra süreç modeline karar verilir (çevik, waterfall, RAD vb.). Süreç bu modele göre planlanır.
- Yazılım konfigürasyon yönetimi (“yazılım düzeni”) ve kalite kontrolü planlanır: Hangi dönüm noktasında hangi ürün çıkacak, ne zaman değişiklik yapıldı, bunlar kaydedilir.
- Kullanılacak yöntemler, gösterimler ve araçlar kararlaştırılır: Ör. hangi UML diyagramları kullanılacak, CASE araçları kullanılacak mı?
- Gider tahmini ve görevlendirme: Hangi süre aralığında hangi kişiler çalışacak, maliyetleri nedir?
- Zamanlama programı: Hangi aralıkta hangi iş yapılacak, hangi işler birbirine bağlı?
- Sistemin işletilmesi ve bakımı için gereken kaynaklar tahmin edilir.
- Müşteri, problem ve geliştirici arasında ortak bir terimler sözlüğü hazırlanır.
- Donanım, yazılım ve insan kaynakları tanımlanır.

**Proje planının ayrıntılı içerik listesi**

1. **Kapsam, hedefler ve sınırlar:**

- Örnek: hastane yazılımı. Yalnızca randevu, kayıt ve doktor işlem raporlama mı? Yatan hasta işlemleri, tanı-tedavi istatistikleri, malzeme yönetimi, laboratuvar yönetimi dahil mi?
- Uygulama alanı (hastane bilgi sistemi) ile bu projenin alanı (şu anki sınırlar) ayrılmalıdır.

2. **Yazılım geliştirme süreç modeli:** Kapsama, uygulamaya, müşteriye, süreye ve kaynaklara uygun model seçilir.
3. **Varsayımlar ve kısıtlar:** İşlevsellik, uygulama koşulları, çalışanlar ve işletmecilik açısından.
4. **Risk analizi:** Riskler nelerdir, hangisi daha önemlidir, nasıl ortadan kaldırılır?
5. **Alt aktiviteler (WBS):** Gereksinim analizi, tasarım, gerçekleştirme, test ve bunların alt başlıkları.
6. **Zaman çizelgesi:** Her alt adımın süresi. Sonra aktiviteler arası ilişkilere göre yeniden düzenlenir (biri bitince diğeri mi başlar, ikisi aynı anda mı yürür?).
7. **İş gücü belirleme:** Her alt aktiviteyi hangi tipte ne kadar kişinin yapacağı.
8. **Görev atama:** Hangi kişilerin atanacağı.
9. **Bütçe:**
   - Hangi ücret grubundan, hangi görev seviyesinden kişinin ne kadar süre çalışacağı.
   - Yeni donanım ve satın alınacak yazılım araçları.
   - Benzer uygulamaları yerinde görmek için yol bütçesi.
   - Malzemeler ve alt yükleniciler.
10. **Kalite planı:** Gözden geçirmelerin zamanı, aralıkları, kimin yapacağı, ne kadar süreceği. Kullanılacak standartlar.

- Proje planının ardından yazılım gereksinim spesifikasyonu (SRS) hazırlanır. Planlama için yapılan kısa gereksinim çalışması yeterli değildir; ayrıca gereksinim analizi aşaması yapılmalıdır.
- İleriki derslerde IEEE proje planı standardı da incelenecek.

### Yazılım geliştirme süreç modelleri

- Temel modeller:
  - Waterfall (çağlayan).
  - V modeli: waterfall'a benzer, her aşamaya karşılık doğrulama (verification) adımları vardır.
  - Prototip modeli.
  - Spiral (sarmal) model: risk yönetimine odaklanır. Hoca bunu waterfall ile prototipin birleşimi olarak tanımladı.
  - RAD (Rapid Application Development).
  - Barok modeli (bir öğrencinin hatırladığı ad, altyazıda böyle geçiyor; kesin değil).
- Özel modeller:
  - Unified Process: Rational Rose ile kullanılmak üzere geliştirilmiştir.
- Çevik modeller: XP (Extreme Programming), Kanban, Scrum.
- WBS'deki alt aktivitelerin dizilişi modele göre değişir: waterfall, spiral, RAD ve Scrum'da farklıdır.

### Yazılım risk yönetimi

- Risk, gelecekte değişimle ortaya çıkabilecek problemlerin seçimlerimizle nasıl önlenebileceğini anlatan kavramdır.
- Değişim kaynakları: müşteri gereksinimleri, geliştirme teknolojileri, bilgisayarlar.
- İki özelliği vardır: belirsizlik ve kayıp.
- Genel kategoriler: proje riskleri (zaman, kaynak), teknik riskler (tasarım, kuruluş, doğrulama, bakım), işletmecilik riskleri (pazarlama, stratejik yönetim, bütçe, satış desteği).

### Ürün büyüklüğü riskleri

- Ürünün tahmin edilen büyüklüğü: satır sayısı (LOC) veya function point, program ve dosya sayısı.
- Önceki ürünlerden ne kadar büyük?
- Ürünün oluşturduğu ve kullandığı veri tabanının büyüklüğü. Daha önce yönetilmemiş büyüklükte mi?
- Kullanıcı sayısı.
- Yeniden kullanılan yazılım miktarı.

**İşletmecilik riskleri**

- Ürünün şirket geliri üzerindeki etkisi. Start-up bir şirket bu projeyle hayata başlıyorsa proje başarısızlığı şirketin kuruluşunu bile engeller.
- Dağıtım tarihinin uygunluğu.
- Birlikte çalışılacak başka sistemler.
- Son kullanıcıların düzeyi.
- Geç teslimin maliyeti. Anlaşma nasıl yapılmış?

**Müşteriyle ilişkili riskler**

- Bütün müşteriler eşit değildir. Gereksinimleri ve kişilikleri farklıdır. Bir kontrol listesi oluşturulabilir.
- Bu müşteriyle daha önce çalışıldı mı?
- Müşterinin gereksinimleri hakkında somut fikri var mı?
- Ara kontrollere ve gözden geçirmelere katılmaya istekli mi? Çevik projelerde müşteri temsilcisi ekibin üyesi gibidir. Diğer modellerde de müşteriden bir temsilci veya grup belirlenebilir.
- Müşteri ürün alanında ve yazılım geliştirmede teknik bilgiye sahip mi?
- Daha önce çalışılmış olmak genellikle riski azaltır. Ama bazen müşteriyle çalışmanın zor olduğu bilinir; bu başlı başına risktir.

**Süreç riskleri**

- Analiz, tasarım ve test rastgele yapılıyorsa proje risk altındadır.
- SEI (Software Engineering Institute) bu konuda sorular hazırlamıştır:
  - Yazılım sürecinin yazılı tanımı var mı? Çalışanlar bunu kullanmaya istekli mi?
  - Süreç tanımı başka projelerde kullanıldı mı?
  - Yöneticilere ve teknik personele yazılım mühendisliği eğitimi verildi mi?
  - Standartlar biliniyor mu?
  - Gözden geçirme sonuçları belgeleniyor mu?
  - Konfigürasyon yönetimi yapılıyor mu?
- “Evet” cevapları arttıkça süreç daha iyi yönetiliyor demektir.

**Teknik riskler**

- Müşteri ile geliştirici arasındaki iletişime yardım eden uygulama spesifikasyon teknikleri kullanılıyor mu?
- Analiz için belirli bir yöntem var mı?
- Kod içinde belgeleme ve açıklama var mı?
- Test case tasarımı için yöntem var mı?
- Analiz ve tasarımda yazılım araçları kullanılıyor mu?
- Prototip oluşturuluyor mu, bunun için araç var mı?

**Personel riskleri**

- En sık karşılaşılan risk türüdür.
- İnsanlar yeteneklerine göre mi kullanılıyor?
- Yeterli çalışan var mı?
- Personel çalışma süresinin ne kadarını projeye ayırıyor?
- Verilen işler çalışanların beklentilerini karşılıyor mu?
- Personel gerekli eğitimi aldı mı?
- Bunların çoğu “hayır” ise personel riski vardır.

**Makale ve uygulama projesi duyuruları**

- Makale konuları dersin kapsamındaki konularda, yazılım proje yönetimiyle ilgili yeni yayınlardan seçilmelidir.
- Önerilen dergiler: IEEE Software, Journal of Systems and Software, IEEE Transactions on Software Engineering, IEEE Transactions on Engineering Management. Mümkün olduğunca 2015 sonrası.

- Kabul edilen önerilerden örnekler (aynı konular tekrar seçilmemeli):
  - Monte Carlo simülasyonuna dayalı yazılım proje takvimi yönetimi (2020 bilgi teknolojileri sempozyumu bildirisi).
  - Çevik yazılım geliştirme projelerinde proje yöneticisi var mı? (2016 makalesi). Hoca, öğrencinin Türkiye için bir anket çalışması yapma isteğini destekleyebileceğini söyledi.
  - Çevik proje yönetiminin kurumda benimsenmesi (adoption) ve kritik başarı faktörleri (2021 tarihli bir derleme makalesi). Hoca, 15-20 yıl önce konfigürasyon yönetiminin kurumda benimsenmesi üzerine benzer bir çalışma yaptığını anlattı.
  - Yazılım projelerinde risk faktörleri (2016 konferans bildirisi).
- **Teslim:** Teklif vermemiş herkes önümüzdeki haftaya kadar bir makale teklifi e-postayla göndermeli. Başlangıç makalesi belirtilmeli.
- Ödevler daha sonra online Yıldız sistemindeki "ödev ve sınavlar" bölümüne yüklenecek.
- Makale ve uygulama projesi iki kişiyle yapılabilir. Makale sunumunda her öğrenci kendi bölümünü anlatmalıdır.
- Uygulama projesi: Bir yazılım projesinin proje planı veya teklif dokümanı hazırlanacak. Diyagramlar, zamanlama ve iş-kaynak çizelgeleri oluşturulacak. Sistemin küçük bir parçası geliştirilecek ve plana uyum kontrol edilecek. Bitirme projesi, iş yerindeki bir iş veya küçük bir proje genişletilebilir. TÜBİTAK formatında bir Ar-Ge teklifi de olabilir.
- Hoca konu anlatımından sonra makale sunumlarının yapılacağını hatırlattı.

**Gelecek ders**

- Risk analizine, kabul ve kısıtlara devam edilecek.
- Zaman çizelgesi ve aktiviteler arası ilişkiler işlenecek.

**Ders sonrası notlar**

- Bir öğrencinin "salyangoz model" dediği model spiral (sarmal) modeldir.
- Altyazıdaki "Reşinal Rose" Rational Rose, "Software Engineer Enstitü" Software Engineering Institute (SEI), "ecel" ise çevik (agile) yaklaşımdır.
- Hocanın andığı "Systems and Software" dergisi Elsevier'in Journal of Systems and Software dergisidir.
- Hoca spiral modeli waterfall ile prototipin birleşimi olarak tanımladı. Boehm'in spiral modeli yinelemeli prototiplemeyi waterfall'un sistematik adımlarıyla birleştirir ve her turda risk analizi içerir. Tanım bu anlamda doğrudur.
- Çevik benimseme makalesinin yayın yeri altyazıda "2021 International System Science" diye geçiyor. Büyük olasılıkla HICSS (Hawaii International Conference on System Sciences) kastediliyor.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.